



Материал о навык... · [Подписаться на публикацию](#)

Кибербезопасность +

Программирование +

Разработка программного обеспечения +

ТЕХНОЛОГИЯ +

Веб-разработка +

# Ошибки в безопасности JWT, которые я до сих пор встречаю в продакшене

9 минут чтения · 27 июля 2026 года



Кодовая настройка

Подписаться



Слушай



Поделиться

*Большинство проблем с JWT связаны не с несовершенством криптографии. Они возникают из-за слабой проверки, устаревших утверждений, небезопасного хранения и поведения сессии, которое никто не продумывал.*



Токен был действительным.

Его подпись совпадала с заявленной. Срок действия не истек. Промежуточное ПО для аутентификации приняло его без замечаний.

Запрос все равно не должен был быть разрешен.

Это противоречие объясняет многие проблемы с JWT, которые я до сих пор наблюдаю в продакшене. Команда проверяет подпись, срок действия и считает, что сложная работа по обеспечению безопасности завершена.

Это не так.

Проверка отвечает на узкий вопрос:

---

Удовлетворяет ли этот токен требованиям криптографической политики и политики проверки утверждений, настроенной приложением?

---

Это не гарантирует автоматически, что учетная запись активна, разрешение актуально, токен был создан для этого API, пользователь владеет запрашиваемым ресурсом или что операция безопасна для выполнения.

JWT — это формат представления утверждений. Он не определяет полную архитектуру аутентификации, авторизации, сессий, обновления, отзыва и выхода из системы на основе этих утверждений.

Главная ошибка — рассматривать технически действительный токен как полноценное решение для обеспечения безопасности.

Токен был действительным.

Но решение все равно было неверным.

## 1. Мы рассматривали проверку подписи как авторизацию

Обычная проверка авторизации выглядит так:

```
const claims = verifyAccessToken(token);
if (claims.role === "admin") {
  await refundPayment(paymentId);
}
```

Токен может быть подписан правильно. Требование `role` может быть выдано доверенной системой авторизации.

Но это все равно не гарантирует, что возврат средств будет осуществлен.

Возможно, API также потребуется следующая информация:

- принадлежит ли платеж организации пользователя;
- предусмотрено ли разрешение для этого действия;
- можно ли вернуть платеж;
- приостановлена ли учетная запись;
- превышает ли сумма лимит пользователя;
- требуется ли для выполнения операции недавняя аутентификация.

Более надежная структура отделяет проверку токенов от авторизации на уровне ресурсов:

```
const identity =
  verifyOrdersAccessToken(token);
const payment =
  await payments.getById(paymentId);
const decision = await refundPolicy.evaluate({
  actorId: identity.subject,
  organizationId: identity.organizationId,
  payment,
});
if (!decision.allowed) {
  throw new ForbiddenError();
}
await payments.refund(payment.id);
```

The token identifies the actor and may carry useful scopes or entitlements. The resource server still combines those claims with current application and resource context.

This also prevents a dangerous mismatch:

```
// Dangerous
await authorizeAccount(
  identity.subject,
  accountId,
);
await refundPayment(paymentId);
```

The code authorizes one object and acts on another.

Authorization should be tied to the exact payment, document, order, tenant, or account being changed.

A signature proves integrity under a trusted key. It does not prove authorization for this request.

## 2. We Let the Token Choose the Verification Policy

A JWT header may contain values such as:

```
{
  "alg": "RS256",
  "kid": "key-2026-01"
}
```

These values help the verifier process the token.

They should not define what the application trusts.

A dangerous design decodes attacker-controlled headers and then follows their instructions:

```
const header = decodeJwtHeader(token);
const key = await resolveKeyRequestedByToken(
  header,
);
return verifyJwt(token, key);
```

The server may accept an unexpected algorithm, retrieve a key from an untrusted location, or use key material in a way the application never intended.

A safer verifier begins with server-controlled expectations:

```
const claims = verifyJwt(
  token,
  trustedAuthorizationKeys,
  {
    allowedAlgorithms: ["RS256"],
    expectedIssuer:
      "https://auth.example.com",
    expectedAudience: "orders-api",
  },
);
```

RS256 is illustrative. The accepted algorithms should match the deployment's explicit token profile and key-management design.

The important direction is consistent:

- the server defines acceptable algorithms;
- the server defines trusted issuers;
- keys come from trusted configuration or metadata associated with a preconfigured trusted issuer;
- kid selects from an approved key set;
- unknown algorithms and keys are rejected;
- token-provided key URLs are not followed blindly.

Key rotation does not require trusting arbitrary material:

```
const key = trustedKeys.find(
  (candidate) =>
    candidate.id === header.kid,
);
if (!key) {
  throw new InvalidTokenError(
    "Unknown signing key",
  );
}
```

Here, kid selects from an already trusted set. It does not give the token permission to introduce a new issuer, key server, or trust source.

The token describes how it was signed.

The server decides which signing methods and keys it trusts.

### 3. We Ignored Audience, Issuer, Type, and Purpose

A token may be authentic and still belong somewhere else.

It may have been issued for:

- another API;
- another environment;
- another client;
- another tenant;
- another token class;
- another operation.

An ID token should not automatically work as an API access token. A password-reset token should not become a session credential. An email-verification token should not authorize ordinary resource requests.

A generic verifier makes substitution easier:

```
function verifyAnyToken(token: string) {  
  return verifyJwt(token, sharedKeys);  
}
```

A better design gives each token profile its own validation policy:

```
function verifyOrdersAccessToken(  
  token: string,  
) {  
  return verifyJwt(token, accessTokenKeys, {  
    allowedAlgorithms: ["RS256"],  
    expectedIssuer: "https://auth.example.com",  
    expectedAudience: "orders-api",  
    expectedType: "at+jwt",  
    requiredClaims: ["sub", "exp"],  
  });  
}
```

The exact `typ` value depends on the token profile being implemented. A custom JWT does not automatically need to use `at+jwt`.

The verifier should define how it handles both JOSE header parameters and JWT claims.

### Header parameters include:

- `typ` : the token class or profile;
- `alg` : the signing or encryption algorithm;
- `kid` : the key identifier.

### Claims include:

- `iss` : who issued the token;
- `aud` : which recipient should accept it;
- `sub` : which subject it represents;
- `exp` : when it expires;
- `nbf` : when it becomes valid;
- `iat` : when it was issued;
- `jti` : its identifier.

Not every claim is universally mandatory. The application or protocol profile must define which values are required and how they are interpreted.

Authentic is not the same as intended for this audience.

A token can pass signature verification and still be valid for the wrong service, environment, or purpose.

## 4. We Treated Claims as Secret and Permanently Current

A typical signed JWT has three parts:

```
header.payload.signature
```



An unencrypted signed JWT should be treated as readable by any party that obtains it.

The signature protects integrity. It allows the receiver to detect modification. It does not hide the payload.

JWTs may also be encrypted as JWEs or use nested signed-and-encrypted forms, but ordinary signed access tokens are not automatically confidential.

That makes this payload dangerous:

```
{
  "sub": "user-123",
  "email": "person@example.com",
  "phone": "+1...",
  "internalNotes": "...",
  "recoveryCode": "...",
  "role": "admin"
}
```

A token should not become a portable copy of the user table.

Prefer only the claims the receiving service genuinely needs:

```
{
  "sub": "user-123",
  "iss": "https://auth.example.com",
  "aud": "orders-api",
  "exp": 1780000000,
  "scope": "orders:read orders:refund"
}
```

Readable does not create a stable client contract.

In OAuth 2.0, clients should treat access tokens as opaque credentials. Under the JWT access-token profile, clients must not inspect their contents or build application behavior around internal claims.

The authorization and resource servers own the token representation and interpretation. They may later replace a JWT access token with an opaque value without changing the client's responsibilities.

Claims also become stale.

Suppose a token contains:

```
{  
  "role": "admin"  
}
```

After issuance, the user may be demoted, disabled, removed from an organization, or required to complete MFA again.

---

Get CodeTune's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

---

The claim remains cryptographically valid until the token expires or another invalidation mechanism intervenes.

This does not mean every request must reload every permission from a database. It means freshness should match risk.

A low-risk read may rely on a short-lived scope. A high-impact action may require current account status, resource ownership, transaction limits, risk context, or recent authentication.

A signed claim is a trusted snapshot.

It is not a permanently current fact.

## 5. We Chose Browser Storage Without a Threat Model

Many browser applications choose token storage based on convenience:

```
localStorage.setItem(  
  "access_token",  
  token,  
);
```

The token is now easy for application JavaScript to retrieve.

It may also be retrievable by malicious JavaScript executing in the same origin.

An HttpOnly cookie reduces direct credential theft because normal page JavaScript cannot read it:

```
Set-Cookie: __Host-session=...; Path=/; HttpOnly; Secure; SameSite=Lax
```

The `__Host-` prefix requires a secure, host-only cookie with `Path=/` and no `Domain` attribute.

The correct `SameSite` policy depends on the application flow. `Strict` can break legitimate cross-site navigation, while `Lax` may be an intentional compromise.

Cookies also introduce CSRF responsibilities because browsers attach matching cookies automatically.

Controls may include:

- an appropriate `SameSite` policy;
- CSRF tokens or Origin validation;
- Fetch Metadata checks;
- no state-changing `GET` endpoints;
- CORS rules that do not permit credentialed access from untrusted origins.

CORS is not a replacement for CSRF protection.

Cross-origin requests may still be sent even when browser JavaScript cannot read their responses.

HttpOnly does not make XSS harmless either. Injected JavaScript may be unable to steal the cookie directly but can still perform authorized actions through the compromised browser.

The storage decision changes which attacks are easier. It does not make the session trustworthy after the page itself has been compromised.

Access tokens should also stay out of URLs:

```
https://app.example.com/dashboard  
?access_token=eyJ...
```

OAuth security best practice prohibits passing access tokens through URI query parameters. URLs can leak credentials through browser history, referrer data, proxy logs, analytics, screenshots, and support systems.

Token storage is not a frontend convenience decision.

It is a threat-model decision involving XSS, CSRF, persistence, replay, and session architecture.

## 6. Refresh, Logout, and Revocation Had No Session Model

A refresh token is not simply an access token with a longer expiration.

It represents longer-lived authority to obtain new access. Theft and replay are therefore especially damaging.

A mature refresh design may require:

- rotation and reuse detection;
- expiration and session binding;
- token-family tracking;

- revocation;
- sender-constrained protection.

Current OAuth security best practice requires refresh tokens issued to public clients to use rotation or sender-constrained protection.

With rotation, the previous refresh token is invalidated when a new one is issued. The authorization server retains enough relationship information to detect suspicious reuse.

Refresh credentials do not have to be JWTs.

An access token may be a JWT while the refresh credential is an opaque random value backed by a server-side session record.

Logout requires the same architectural clarity.

This removes one local copy:

```
localStorage.removeItem("access_token");
```

It does not invalidate:

- a token already copied by an attacker;
- a refresh token stored elsewhere;
- the same session on another device;
- an access token still accepted by an API;
- a credential present in logs or extensions.

A meaningful logout may revoke refresh credentials, terminate the server-side session, and define what happens to already-issued access tokens.

Short-lived access tokens can limit the remaining window. Immediate invalidation may require denylisting, token versions, introspection, or another server-side check.

Systems requiring device logout, reuse detection, incident response, or immediate logout usually retain some state.

JWT can reduce per-request session lookup.

It does not eliminate session lifecycle design.

Stateless access does not mean stateless session management.

## 7. We Protected Tokens in Code and Leaked Them Through Operations

A token can be generated, signed, stored, and verified correctly, then leak through operational tooling.

Common exposure points include:

- debug and proxy logs;
- exception reports;
- distributed tracing;
- analytics payloads;
- URLs and screenshots;
- copied authorization headers.

This is dangerous:

```
logger.error("JWT validation failed", {  
    token,  
    error,  
});
```

The log entry now contains a reusable credential.

A safer event records evidence about the failure without retaining the token:

```
logger.warn("JWT validation failed", {  
    reason: "invalid-audience",
```

```
expectedAudience: "orders-api",
issuer: safeIssuer,
tokenFingerprint:
  createHmacFingerprint(
    token,
    loggingFingerprintKey,
  ),
requestId,
});
```

A correlation fingerprint should use a dedicated key separate from signing and encryption keys. The output can be truncated to the amount needed for investigation, and log access should remain restricted.

Raw attacker-controlled values such as `kid`, `iss`, or error text should also be validated, encoded, or normalized before entering logs. Logging systems can become injection targets too.

Useful security events may include:

- unexpected algorithms or keys;
- invalid signatures, issuers, or audiences;
- wrong token types;
- expired or not-yet-valid tokens;
- refresh-token reuse;
- revoked-session use;
- repeated authorization failures.

Logging only this may provide too little evidence:

```
logger.warn("Invalid token");
```

Logging this creates a new credential leak:

```
logger.warn(`Invalid token: ${token}`);
```

The goal is structured telemetry that supports investigation without becoming another credential store.

Logs should capture the security event without capturing the credential.

## **A JWT Is Only as Secure as the Trust Model That Consumes It**

I used to review JWT code by looking for signature verification and expiration checks.

Now I ask different questions.

Which algorithms and keys does the server explicitly trust?

Who issued the token?

Which API was it created for?

What kind of token is it?

Are its permissions still acceptable for this operation?

Does the user own the resource being changed?

Where can the credential be stolen?

How does refresh rotation work?

What does logout invalidate?

How quickly can a compromised session be revoked?

Can a raw token appear in logs, URLs, traces, or analytics?

JWT validation proves that a token satisfied a configured trust policy.

Authorization still has to prove that the current actor, resource, action, session, and context make the request acceptable now.



The signature matters.

The trust model matters more.

The most dangerous JWT is not always invalid. Sometimes it is perfectly valid and trusted for more than it was ever meant to prove.

Cybersecurity +

Programming +

Software Development +

Technology +

Web Development +



Follow

## Published in Skill Stuff

781 followers · Last published just now

Skill Stuff is your go-to hub for coding tips, problem-solving strategies, and developer growth insights.



Follow

## Written by CodeTune

285 followers · 2.4K following

Full-stack engineer turning complex ideas into simple, high-performance products.



